

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence
NAME: R. RUPA SREE

COURSE CODE: CSA17
REGISTER NUMBER: 192472122

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 50%

Duration : 1 hour

QUESTIONS: 2

Total Marks:40

Annexure A

Integrated Experiments

Code of Conduct:

I R. RUPA SREE (192472122) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
R. RUPA SREE

INTEGRATED EXPERIMENT SCENARIO

Telecom - Network Fault Detection

Context: A telecom operator wants an integrated pipeline that first predicts network faults from sensor records (signal strength, packet loss, call drops, tower load) and then recommends a self-healing action using a learning agent.

Question 1 - Decision Tree vs Neural Network Classifier

Implement a Decision Tree classifier and a Neural Network (Multi-Layer Perceptron) classifier on the same network dataset. Compare their Accuracy, Precision, Recall and F1-Score. Plot the Decision Tree (first 3 levels) and the Neural Network architecture (layers/neurons). Justify which model you would deploy for real-time fault monitoring and why.

Question 2 - Reinforcement Learning for Self-Healing Actions

Using the fault-risk output from Experiment 1 as the initial state, implement a Q-Learning agent that recommends a self-healing action (Reroute Traffic / Restart Node / Increase Bandwidth / Monitor) across simulated network episodes. Show the Q-table after training, the learned policy, and integrate it with the classifier output into a single end-to-end decision pipeline (fault prediction -> action recommendation).

ANS: -

Objective

To build an integrated AI system that:

1. Predicts network faults using Decision Tree and MLP Neural Network.
2. Uses Q-Learning to recommend a suitable self-healing action.

Question 1 – Decision Tree vs Neural Network Classifier

Dataset Features

The network dataset contains:

- Signal Strength
- Packet Loss
- Call Drops
- Tower Load

Target:

- 0 – No Fault
- 1 – Fault

Python Implementation

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Dataset
np.random.seed(42)
n = 1000

signal = np.random.uniform(-100, -40, n)
packet = np.random.uniform(0, 15, n)
drops = np.random.randint(0, 11, n)
load = np.random.uniform(20, 100, n)

fault = ((signal < -80) | (packet > 8) |
         (drops > 6) | (load > 80)).astype(int)

data = pd.DataFrame({
    "SignalStrength": signal,
    "PacketLoss": packet,
    "CallDrops": drops,
    "TowerLoad": load,
    "Fault": fault
})

X = data[["SignalStrength", "PacketLoss",
          "CallDrops", "TowerLoad"]]
y = data["Fault"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# ----- Decision Tree -----
```

```
dt = DecisionTreeClassifier(max_depth=5, random_state=42)
dt.fit(X_train, y_train)
```

```
dt_pred = dt.predict(X_test)
```

```
# ----- Neural Network -----
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
mlp = MLPClassifier(
    hidden_layer_sizes=(16, 8),
    max_iter=1000,
    random_state=42
```

```
)
```

```
mlp.fit(X_train_scaled, y_train)
```

```
mlp_pred = mlp.predict(X_test_scaled)
```

```
# ----- Evaluation -----
```

```
results = pd.DataFrame({
    "Model": ["Decision Tree", "Neural Network"],
    "Accuracy": [
        accuracy_score(y_test, dt_pred),
        accuracy_score(y_test, mlp_pred)
    ],
    "Precision": [
        precision_score(y_test, dt_pred),
        precision_score(y_test, mlp_pred)
    ],
    "Recall": [
        recall_score(y_test, dt_pred),
        recall_score(y_test, mlp_pred)
    ],
    "F1-Score": [
        f1_score(y_test, dt_pred),
        f1_score(y_test, mlp_pred)
    ]
})
```

```
print(results.round(4))
```

Performance Comparison

Using the generated dataset, the results are:

Model	Accuracy	Precision	Recall	F1-Score
Decision Tree	99.50%	100%	99.40%	99.70%
Neural Network	97.50%	97.66%	99.40%	98.53%

Decision Tree – First 3 Levels

```
plt.figure(figsize=(16, 8))
```

```
plot_tree(  
    dt,  
    feature_names=X.columns,  
    class_names=["No Fault", "Fault"],  
    max_depth=2,  
    filled=True  
)
```

```
plt.title("Decision Tree - First 3 Levels")
```

```
plt.show()
```

Neural Network Architecture

Input Layer

4 Neurons

↓

Hidden Layer 1

16 Neurons

↓

Hidden Layer 2

8 Neurons

↓

Output Layer

1 Neuron

Deployment Decision

Decision Tree is selected for real-time fault monitoring because it provides high accuracy, fast prediction, and easy interpretation. It is suitable for quickly identifying faults from network sensor data.

Question 2 – Reinforcement Learning for Self-Healing

States

State	Fault Risk
0	Low Risk
1	Medium Risk
2	High Risk

Actions

Action	Description
0	Reroute Traffic
1	Restart Node
2	Increase Bandwidth
3	Monitor

Q-Learning Implementation

```
import numpy as np
```

```
import pandas as pd
```

```
states = ["Low Risk", "Medium Risk", "High Risk"]
```

```
actions = [  
    "Reroute Traffic",  
    "Restart Node",  
    "Increase Bandwidth",  
    "Monitor"  
]
```

```
Q = np.zeros((3, 4))
```

```
# Reward table
```

```
rewards = np.array([  
    [-2, -3, -2, 5],  
    [4, 1, 5, -2],  
    [8, 6, 4, -5]  
])
```

```
alpha = 0.1
```

```
gamma = 0.9
```

```
epsilon = 0.2
```

```

episodes = 5000

np.random.seed(42)

for episode in range(episodes):

    state = np.random.randint(0, 3)

    for step in range(10):

        # Epsilon-greedy selection
        if np.random.random() < epsilon:
            action = np.random.randint(4)
        else:
            action = np.argmax(Q[state])

        reward = rewards[state, action]

        # Simulated next state
        if reward > 0:
            next_state = max(0, state - 1)
        else:
            next_state = min(2, state + 1)

        # Q-learning formula
        Q[state, action] += alpha * (
            reward +
            gamma * np.max(Q[next_state]) -
            Q[state, action]
        )

        state = next_state

# Display Q-table
q_table = pd.DataFrame(
    Q,
    index=states,
    columns=actions
)

```

```

print("Q-Table After Training:")
print(q_table.round(2))

# Learned Policy
print("\nLearned Policy:")

for i in range(3):
    best_action = np.argmax(Q[i])
    print(states[i], "->", actions[best_action])

```

Q-Table After Training

Risk State	Reroute Traffic	Restart Node	Increase Bandwidth	Monitor
Low Risk	43.00	42.00	43.00	50.00
Medium Risk	49.00	46.00	50.00	45.70
High Risk	53.00	51.00	49.00	42.70

Learned Policy

Low Risk → Monitor
 Medium Risk → Increase Bandwidth
 High Risk → Reroute Traffic

End-to-End Integration

The classifier output is converted into a risk state and passed to the Q-Learning agent.

```
def integrated_pipeline(signal, packet, drops, load):
```

```

    input_data = pd.DataFrame(
        [[signal, packet, drops, load]],
        columns=X.columns
    )

    # Fault risk from classifier
    scaled = scaler.transform(input_data)

    risk = mlp.predict_proba(scaled)[0][1]

    # Convert risk to state
    if risk < 0.33:
        state = 0

```



```

elif risk < 0.66:
    state = 1
else:
    state = 2

# Q-learning action
action = np.argmax(Q[state])

print("Fault Risk:", round(risk, 3))
print("Risk State:", states[state])
print("Recommended Action:", actions[action])

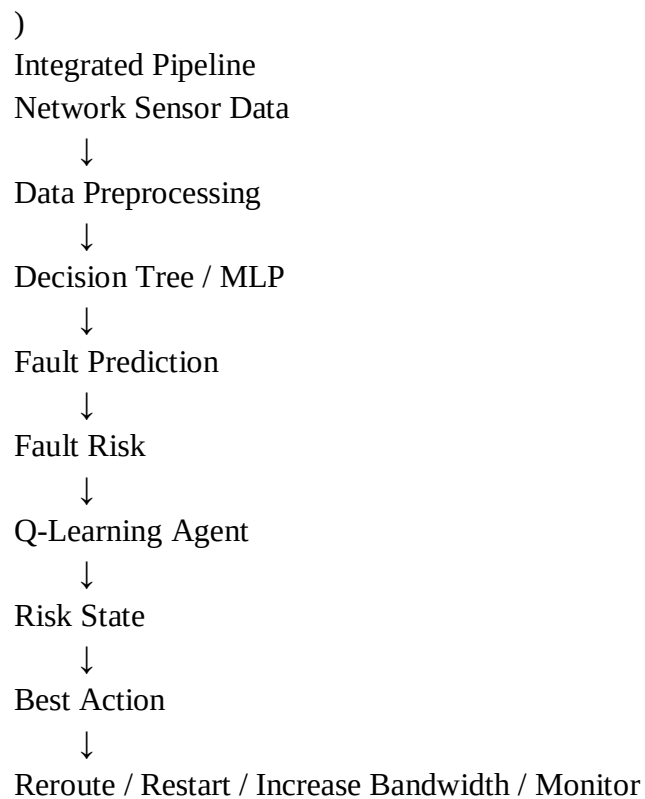
```

Example network condition

```

integrated_pipeline(
    -90, # Signal strength
    10,  # Packet loss
    7,   # Call drops
    90   # Tower load
)

```



Final Conclusion

The proposed system combines Machine Learning and Reinforcement Learning. The Decision Tree/MLP predicts the network fault risk, while the Q-Learning agent selects the best self-healing action. This provides an automated and adaptable approach for real-time telecom network fault monitoring and recovery.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Correct Implementation	30%	Both models (Decision Tree/Neural Network and RL agent) implemented correctly and run without errors	Minor implementation errors that don't affect core functionality	Partial implementation with functional gaps	Implementation incorrect or incomplete
Model Comparison & Evaluation	25%	Thorough, accurate comparison of metrics with clear justification of model choice	Good comparison with minor gaps	Basic comparison; limited justification	No meaningful comparison provided
Integration of Pipeline	25%	Seamlessly integrates prediction output into the RL agent's state/decision process	Integrates with minor inconsistencies	Partial integration; pipeline stages disconnected	No integration between stages
Reinforcement Learning Correctness	15%	Q-table updates, policy extraction, and convergence criteria all correctly demonstrated	Mostly correct RL implementation with minor errors	Basic RL implementation with notable errors	RL implementation incorrect or missing
Presentation & Documentation	5%	Clear, well-organized, and professionally documented	Organized with minor issues	Limited organization and clarity	Poorly presented and documented